

Software Requirements Specification (SRS)

Project: Vehicle Parking System

Version: 1.0

Date: 03-09-2026

Status: Draft — For Submission

Authors: Team 10:

Rohan Rajeev Chirbi - PES2UG24CS408

Rohith S – PES2UG24CS409

Ronak Thamaran - PES2UG24CS410

S Adithya Singh - PES2UG24CS412

Revision history

Version	Date	Author	Change summary	Approval
1.0	03-09-2026	Team 10	Initial SRS draft	Pending

Approvals

Role	Name	Signature / Email	Date
Course Coordinator			

Table of Contents

1. Introduction
2. Overall description
3. External interface requirements
4. System features (detailed)
5. Non-functional requirements (detailed)
6. Quality attributes & Acceptance tests
7. System models and diagrams
8. Requirements Traceability Matrix (RTM)

1. Introduction

1.1 Purpose

This document specifies the functional and non-functional requirements for the Vehicle Parking System (VPS), a web-based application for managing parking slot allocation and vehicle entry/exit and occupancy tracking. It is intended for developers, evaluators, and course instructors as a reference for design, implementation, and assessment.

1.2 Scope

The system covers user registration and login, searching and reserving parking slots, logging vehicle entry and exit, automated fee calculation, and payment recording. It excludes physical hardware integration (e.g., boom barriers, ANPR cameras) beyond a simulated/manual data-entry interface and does not cover multi-city or franchise-level administration.

1.3 Audience

Developers, QA testers, project evaluators, and end users (vehicle owners and parking lot administrators).

1.4 Definitions, Acronyms, and Abbreviations

- VPS — Vehicle Parking System
- Python/Flask — MongoDB, Express.js, React.js, Node.js
- API — Application Programming Interface
- UI — User Interface
- ANPR — Automatic Number Plate Recognition
- JWT — JSON Web Token
- CRUD — Create, Read, Update, Delete

2. Overall description

2.1 Product perspective

VPS is a standalone web application built using Python with the Flask framework and SQLite database. It consists of a client-facing web UI, a backend REST API, and a database for storing users, slots, vehicles, and transactions.

2.2 Major product functions

- User registration and authentication
- View current parking slot availability
- Reserve/book a parking slot in advance
- Record vehicle entry with slot assignment
- Record vehicle exit and compute parking duration
- Automated fee calculation based on duration/vehicle type
- Payment recording and receipt generation
- Admin dashboard for managing slots, rates, and viewing reports

2.3 User roles and characteristics

- Vehicle Owner/User: books slots, views availability, makes payments; assumed to have basic web/app literacy.
- Parking Lot Admin/Operator: manages slot inventory, pricing, and monitors occupancy; needs a simple operational dashboard.
-

2.4 Operating environment

Client: any modern web browser (Chrome, Firefox, Edge) on desktop or mobile. Server: Python with Flask hosted on a suitable local or cloud environment. Database: SQLite.

2.5 Constraints

- Must be implemented within a single academic semester using Python with the Flask framework.
- No integration with real hardware (barriers, sensors, cameras); entry/exit is simulated via UI actions.
- Team size and available development time limit scope to core features only.

3. External interface requirements

3.1 User interfaces

A responsive web UI with pages for login/registration, slot search, booking, entry/exit logging, payment, and an admin dashboard. Clear error messages and confirmation dialogs for booking/payment actions.

3.2 Hardware interfaces

None required for the base system. Optional future extension: ANPR camera or IoT sensor input for automated entry/exit detection.

3.3 Software interfaces

- Backend REST API using Flask exposing endpoints for authentication, slots, bookings, and payments.
- Database interface: SQLite using Python's built-in sqlite3 module.
- No external payment gateway is required; payments are simulated within the system.

3.4 Communications

HTTPS for all client-server communication. JSON as the data interchange format between frontend and backend. JWT-based token authentication for session management.

4. System features (detailed)

Each requirement includes acceptance criteria and a reference test case. IDs follow PVT-F-###.

4.1 User Authentication

Description: Allow users to register, log in, and manage their profile securely.

Req ID	Requirement (shall...)	Type	Priority	Source/Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
PVT-F-001	The system shall allow a new user to register using name, email, and password.	Functional	High	User	AC-F-001: Valid registration creates account and redirects to login. Test: TC-Auth-01	Password shall be hashed (bcrypt).
PVT-F-002	The system shall authenticate registered users via email and password and issue a JWT session token.	Functional	High	User	AC-F-002: Valid credentials return token; invalid credentials show error. Test: TC-Auth-02	Depends on PVT-F-001
PVT-F-003	The system shall allow users to log out and remove their active authentication token.	Functional	Medium	User	AC-F-003: Logout clears token; protected routes become inaccessible. Test: TC-Auth-03	

4.2 Slot Search and Reservation

Description: Enable users to view current slot availability and reserve a slot.

Req ID	Requirement (shall...)	Type	Priority	Source/Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
PVT-F-004	The system shall display real-time availability of parking slots by location/zone.	Functional	High	User	AC-F-004: UI reflects current occupied/free status. Test: TC-Slot-01	
PVT-F-005	The system shall allow filtering of slots by vehicle type (two-wheeler/four-wheeler).	Functional	Medium	User	AC-F-005: Filtered results match selected type. Test: TC-Slot-02	
PVT-F-006	The system shall prevent booking of an already-occupied or already-reserved slot.	Functional	High	User	AC-F-006: Duplicate booking attempt returns conflict error. Test: TC-Slot-03	
PVT-F-007	The system shall allow a logged-in user to reserve an available slot for a specified time window.	Functional	High	User	AC-F-007: Successful reservation marks slot as reserved and returns confirmation. Test: TC-Slot-04	Depends on PVT-F-002

4.3 Vehicle Entry/Exit Tracking

Description: Log vehicle entry and exit events and track slot occupancy duration.

Req ID	Requirement (shall...)	Type	Priority	Source/Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
PVT-F-008	The system shall record vehicle entry details (vehicle number, slot ID, entry timestamp).	Functional	High	Operator	AC-F-008: Entry record created and slot status updated to occupied. Test: TC-Track-01	
PVT-F-009	The system shall record vehicle exit details (exit timestamp) and mark the slot as free.	Functional	High	Operator	AC-F-009: Exit record created; slot status reverts to available. Test: TC-Track-02	Depends on PVT-F-008

Req ID	Requirement (shall...)	Type	Priority	Source/Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
PVT-F-010	The system shall maintain a live occupancy count per parking zone.	Functional	High	Operator	AC-F-010: Occupancy count matches sum of occupied slots. Test: TC-Track-03	
PVT-F-011	The system shall log all entry/exit events with timestamps for audit purposes.	Functional	Medium	Admin	AC-F-011: Logs retrievable via admin view with correct timestamps. Test: TC-Track-04	

4.4 Fee Calculation and Payment

Description: Compute parking fees based on duration and vehicle type, and record payment.

Req ID	Requirement (shall...)	Type	Priority	Source/Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
PVT-F-012	The system shall calculate parking fee based on duration (hourly rate) and vehicle type.	Functional	High	Business	AC-F-012: Fee = rate × billed hours; matches configured rate table. Test: TC-Bill-01	Depends on PVT-F-009 and PVT-F-016
PVT-F-013	The system shall display the calculated fee to the user before exit confirmation.	Functional	High	User	AC-F-013: Fee shown matches backend calculation. Test: TC-Bill-02	Depends on PVT-F-012
PVT-F-014	The system shall record a payment transaction and generate a receipt upon successful payment.	Functional	High	User	AC-F-014: Receipt generated with transaction ID, amount, and timestamp. Test: TC-Pay-01	Depends on PVT-F-013

4.5 Admin Dashboard

Description: Provide administrative controls for slot, rate, and report management.

Req ID	Requirement (shall...)	Type	Priority	Source/Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
PVT-F-015	The system shall allow the admin to add, edit, or remove parking slots.	Functional	Medium	Admin	AC-F-015: CRUD operations reflected immediately in slot listing. Test: TC-Admin-01	
PVT-F-016	The system shall allow the admin to configure hourly parking rates per vehicle type.	Functional	Medium	Admin	AC-F-016: Updated rate applied to new fee calculations. Test: TC-Admin-02	
PVT-F-017	The system shall generate a daily/monthly occupancy and revenue report for the admin.	Functional	Low	Admin	AC-F-017: Report totals match sum of individual transactions. Test: TC-Admin-03	

5. Non-functional requirements (detailed)

NFRs below are measurable and tied to test plans. IDs PVT-NF-####.

Req ID	Requirement	Category	Priority	Acceptance criteria / Measurement
PVT-NF-001	The 90th percentile page load and API response time shall be ≤ 2 seconds under normal load.	Performance	High	Load test shows 90th percentile ≤ 2 s. Test: TC-Perf-01
PVT-NF-002	The system shall support at least 50 concurrent users without degradation.	Scalability	Medium	Load test with 50 concurrent sessions shows stable response. Test: TC-Perf-02
PVT-NF-003	The system shall be available 99% of the time during operational hours.	Reliability	Medium	Uptime monitoring logs $\geq 99\%$ availability. Test: Ops report
PVT-NF-004	The UI shall be usable on both desktop and mobile screen sizes (responsive design).	Usability	Medium	Manual test across 3 screen sizes shows correct layout. Test: TC-UX-01
PVT-NF-005	All passwords shall be stored using a one-way hash (e.g., bcrypt) and never in plaintext.	Security	High	DB inspection confirms hashed storage only. Test: TC-Sec-01

5.1 Security

5.1.1 Security Objectives

- Protect user credentials and personal data from unauthorized access or disclosure.
- Ensure only authenticated and authorized users can perform booking, payment, and admin actions.
- Maintain integrity of transaction and occupancy records against tampering.

5.1.2 Security Requirements

Req ID	Requirement (shall...)	Type	Priority	Acceptance criteria / Test case ref
PRJ-SR-001	The system shall hash and salt all user passwords; plaintext passwords shall never be stored.	Security	High	AC-SR-001: DB inspection shows only hashed passwords. Test: TC-Sec-01
PRJ-SR-002	The system shall enforce role-based access control (Admin vs. User) for all API endpoints.	Security	High	AC-SR-002: Unauthorized role access returns 403. Test: TC-Sec-02
PRJ-SR-003	The system shall use HTTPS/TLS for all client-server communication.	Security	High	AC-SR-003: Network capture confirms TLS in use. Test: TC-Sec-03
PRJ-SR-004	The system shall validate and sanitize all user inputs to prevent SQL/NoSQL injection and XSS.	Security	High	AC-SR-004: Penetration test shows no injection vulnerability. Test: TC-Sec-04
PRJ-SR-005	The system shall lock a user account after 5 consecutive failed login attempts.	Security	Medium	AC-SR-005: 6th failed attempt blocks login for 15 min. Test: TC-Sec-05

6. Quality attributes & Acceptance tests

Exit criteria for acceptance

All high-priority functional requirements implemented and verified; no critical NFR failures; RTM shows all test cases passed.

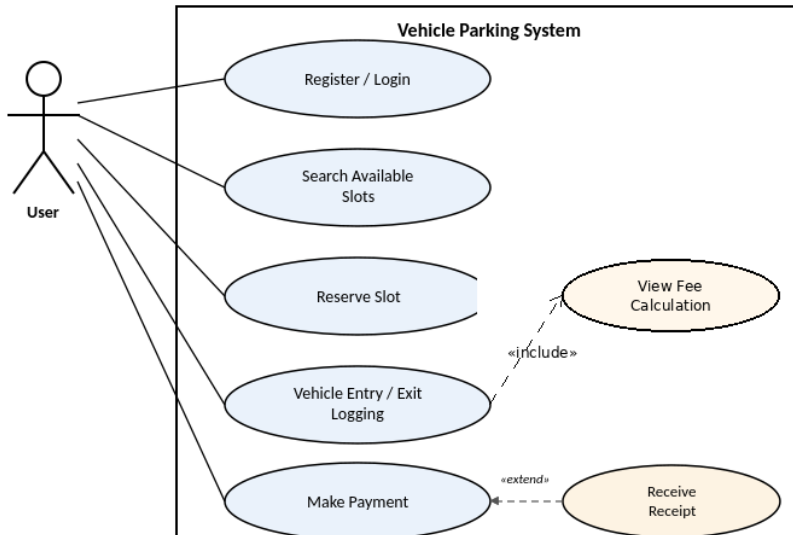
Acceptance test suites

- Authentication (registration, login, logout)
- Slot search and reservation
- Vehicle entry/exit tracking
- Fee calculation and payment
- Admin dashboard operations
- Performance and security tests

7. System models and diagrams

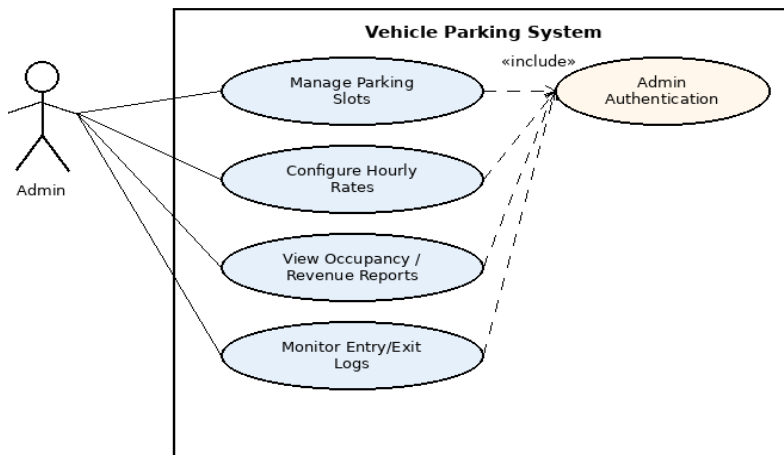
7.1 UML Use-Case Diagram

Use-Case Diagram 1: User interactions



The User actor registers/logs in, searches available slots, reserves a slot, and manages vehicle entry/exit logging. Vehicle entry/exit logging includes viewing the fee calculation, while making a payment extends to receiving a receipt.

Use-Case Diagram 2: Admin interactions



The Admin manages parking slots, configures hourly rates, views occupancy/revenue reports, and monitors entry/exit logs. Each protected admin operation includes admin authentication.

8. Requirements Traceability Matrix (RTM)

Req ID	Requirement (short)	Section ref	Module	Test case(s)	Status (N/P/A)	Comments
PVT-F-001	User registration	4.1	AuthModule	TC-Auth-01	N	
PVT-F-002	User authentication / login	4.1	AuthModule	TC-Auth-02	N	
PVT-F-003	User logout	4.1	AuthModule	TC-Auth-03	N	
PVT-F-004	View available slots	4.2	SlotModule	TC-Slot-01	N	
PVT-F-005	Filter slots by vehicle type	4.2	SlotModule	TC-Slot-02	N	
PVT-F-006	Prevent duplicate slot booking	4.2	SlotModule	TC-Slot-03	N	
PVT-F-007	Reserve a parking slot	4.2	SlotModule	TC-Slot-04	N	
PVT-F-008	Vehicle entry logging	4.3	TrackingModule	TC-Track-01	N	
PVT-F-009	Vehicle exit / slot release	4.3	TrackingModule	TC-Track-02	N	
PVT-F-010	Live occupancy count	4.3	TrackingModule	TC-Track-03	N	
PVT-F-011	Entry/exit audit logs	4.3	TrackingModule	TC-Track-04	N	
PVT-F-012	Parking fee calculation	4.4	BillingModule	TC-Bill-01	N	
PVT-F-013	Display calculated fee	4.4	BillingModule	TC-Bill-02	N	
PVT-F-014	Payment and receipt generation	4.4	PaymentModule	TC-Pay-01	N	
PVT-F-015	Manage parking slots	4.5	AdminModule	TC-Admin-01	N	
PVT-F-016	Configure hourly parking rates	4.5	AdminModule	TC-Admin-02	N	
PVT-F-017	Occupancy/revenue reports	4.5	ReportModule	TC-Admin-03	N	
PVT-NF-001	Response time target	5	WebUI / API	TC-Perf-01	N	
PVT-NF-002	50 concurrent users	5	WebUI / API	TC-Perf-02	N	
PVT-NF-003	99% availability	5	System	Ops report	N	
PVT-NF-004	Responsive desktop/mobile UI	5	WebUI	TC-UX-01	N	
PVT-NF-005	Password hashing	5	AuthModule	TC-Sec-01	N	
PRJ-SR-001	Password hashing and salting	5.1.2	AuthModule	TC-Sec-01	N	
PRJ-SR-002	Role-based access control	5.1.2	SecurityModule	TC-Sec-02	N	
PRJ-SR-003	HTTPS/TLS communication	5.1.2	SecurityModule	TC-Sec-03	N	
PRJ-SR-004	Input validation / injection protection	5.1.2	SecurityModule	TC-Sec-04	N	
PRJ-SR-005	Failed-login account lockout	5.1.2	AuthModule	TC-Sec-05	N	